

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL
数据库

聚合
内存管理
树索引

排序
查询规划
索引构建

并发控制
查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击**底部菜单栏**
称为 **AI 内容创作者**？回复 [添砖加瓦]

21

Database Recovery



Intro to Database Systems
15-445/15-645
Fall 2019

AP

Andy Pavlo
Computer Science
Carnegie Mellon University

CRASH RECOVERY

Recovery algorithms are techniques to ensure database consistency, transaction atomicity, and durability despite failures.

Recovery algorithms have two parts:

- Actions during normal txn processing to ensure that the DBMS can recover from a failure.
- Actions after a failure to recover the database to a state that ensures atomicity, consistency, and durability.

Today

ARIES

Algorithms for Recovery and Isolation Exploiting Semantics

Developed at IBM Research in early 1990s for the DB2 DBMS.

Not all systems implement ARIES exactly as defined in this paper but they're close enough.

ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging

C. MOHAN
IBM Almaden Research Center
and
DON HADERLE
IBM Santa Teresa Laboratory
and
BRUCE LINDSAY, HAMID PIRAHESH and PETER SCHWARZ
IBM Almaden Research Center

In this paper we present a simple and efficient method, called ARIES (*Algorithm for Recovery and Isolation Exploiting Semantics*), which supports partial rollbacks of transactions, fine-granularity (e.g., record) locking and recovery using write-ahead logging (WAL). We introduce the paradigm of *repeating history* to redo all missing updates before performing the rollbacks of the loser transactions during restart after a system failure. ARIES uses a log sequence number in each page to correlate the state of a page with respect to logged updates of that page. All updates of a transaction are logged, including those performed during rollbacks. By appropriate chaining of the log records written during rollbacks to those written during forward progress, a bounded amount of logging is ensured during rollbacks even in the face of repeated failures during restart or of nested rollbacks. We deal with a variety of features that are very important in building and operating an *industrial-strength* transaction processing system. ARIES supports fuzzy checkpoints, selective and deferred restart, fuzzy image copies, media recovery, and high concurrency lock modes (e.g., increment/decrement) which exploit the semantics of the operations and require the ability to perform operation logging. ARIES is flexible with respect to the kinds of buffer management policies that can be implemented. It supports objects of varying length efficiently. By enabling parallelism during restart, page-oriented redo, and logical undo, it enhances concurrency and performance. We show why some of the System R paradigms for logging and recovery, which were based on the shadow page technique, need to be changed in the context of WAL. We compare ARIES to the WAL-based recovery methods of

Authors' addresses: C. Mohan, Data Base Technology Institute, IBM Almaden Research Center, San Jose, CA 95120; D. Haderle, Data Base Technology Institute, IBM Santa Teresa Laboratory, San Jose, CA 95150; B. Lindsay, H. Pirahesh, and P. Schwarz, IBM Almaden Research Center, San Jose, CA 95120.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

© 1992 0362-5915/92/0300-0094 \$1.50

ACM Transactions on Database Systems, Vol. 17, No. 1, March 1992, Pages 94-162

ARIES – MAIN IDEAS

Write-Ahead Logging:

- Any change is recorded in log on stable storage before the database change is written to disk.
- Must use **STEAL** + **NO-FORCE** buffer pool policies.

Repeating History During Redo:

- On restart, retrace actions and restore database to exact state before crash.

Logging Changes During Undo:

- Record undo actions to log to ensure action is not repeated in the event of repeated failures.

TODAY'S AGENDA

Log Sequence Numbers

Normal Commit & Abort Operations

Fuzzy Checkpointing

Recovery Algorithm



WAL RECORDS

We need to extend our log record format from last class to include additional info.

Every log record now includes a globally unique *log sequence number* (LSN).

Various components in the system keep track of *LSNs* that pertain to them...

LOG SEQUENCE NUMBERS

Name	Where	Definition
flushedLSN	Memory	Last LSN in log on disk
pageLSN	$page_x$	Newest update to $page_x$
recLSN	$page_x$	Oldest update to $page_x$ since it was last flushed
lastLSN	T_i	Latest record of txn T_i
MasterRecord	Disk	LSN of latest checkpoint

WRITING LOG RECORDS

Each data page contains a **pageLSN**.

→ The **LSN** of the most recent update to that page.

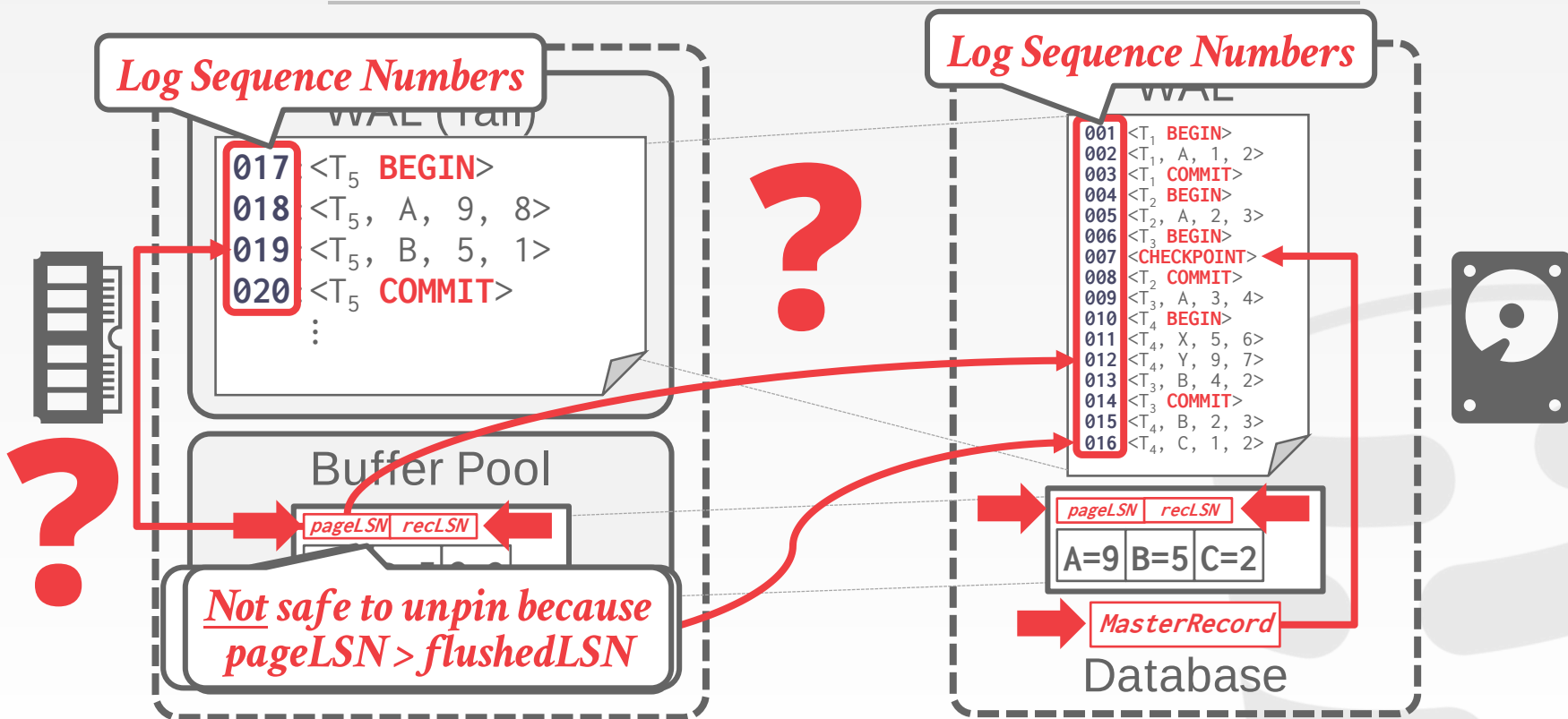
System keeps track of **flushedLSN**.

→ The max **LSN** flushed so far.

Before page **x** can be written to disk, we must flush log at least to the point where:

→ **pageLSN_x ≤ flushedLSN**

WRITING LOG RECORDS



WRITING LOG RECORDS

All log records have an *LSN*.

Update the **pageLSN** every time a txn modifies a record in the page.

Update the **flushedLSN** in memory every time the DBMS writes out the WAL buffer to disk.

NORMAL EXECUTION

Each txn invokes a sequence of reads and writes, followed by commit or abort.

Assumptions in this lecture:

- All log records fit within a single page.
- Disk writes are atomic.
- Single-versioned tuples with Strict 2PL.
- **STEAL** + **NO-FORCE** buffer management with WAL.

TRANSACTION COMMIT

Write **COMMIT** record to log.

All log records up to txn's **COMMIT** record are flushed to disk.

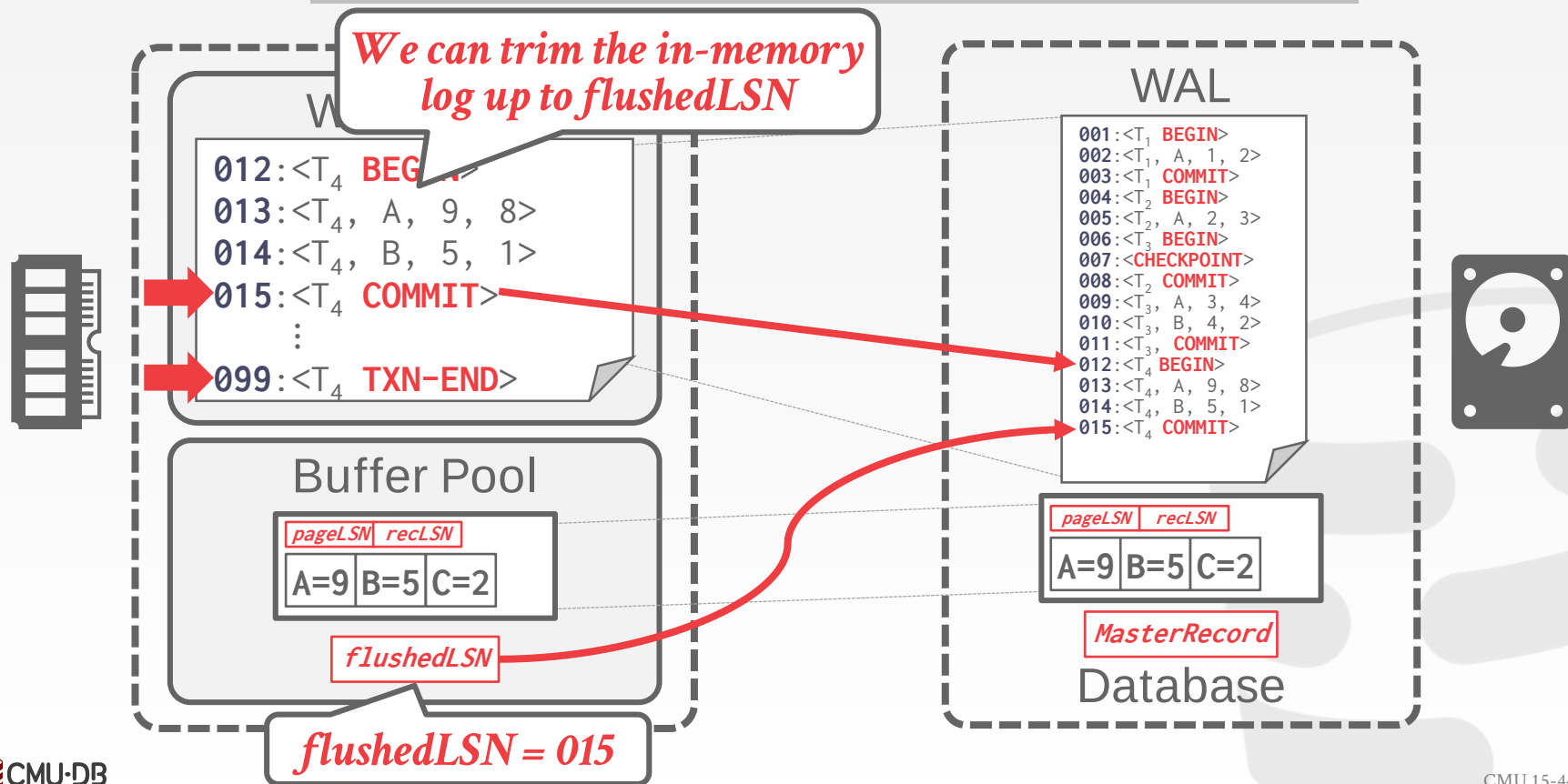
- Note that log flushes are sequential, synchronous writes to disk.
- Many log records per log page.

When the commit succeeds, write a special **TXN-END** record to log.

- This does not need to be flushed immediately.

TRANSACTION COMMIT

We can trim the in-memory log up to flushedLSN



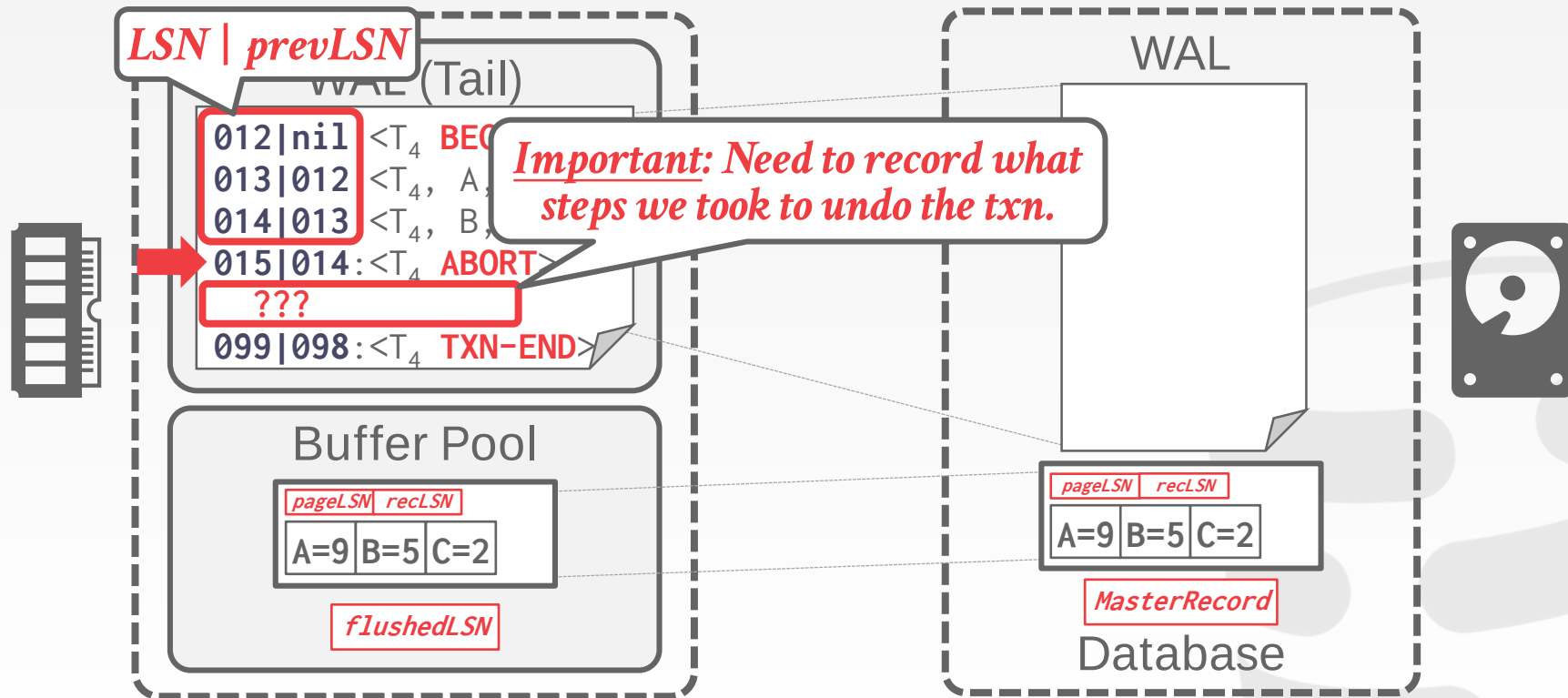
TRANSACTION ABORT

Aborting a txn is actually a special case of the ARIES undo operation applied to only one transaction.

We need to add another field to our log records:

- **prevLSN**: The previous *LSN* for the txn.
- This maintains a linked-list for each txn that makes it easy to walk through its records.

TRANSACTION ABORT



COMPENSATION LOG RECORDS

A **CLR** describes the actions taken to undo the actions of a previous update record.

It has all the fields of an update log record plus the **undoNext** pointer (the next-to-be-undone LSN).

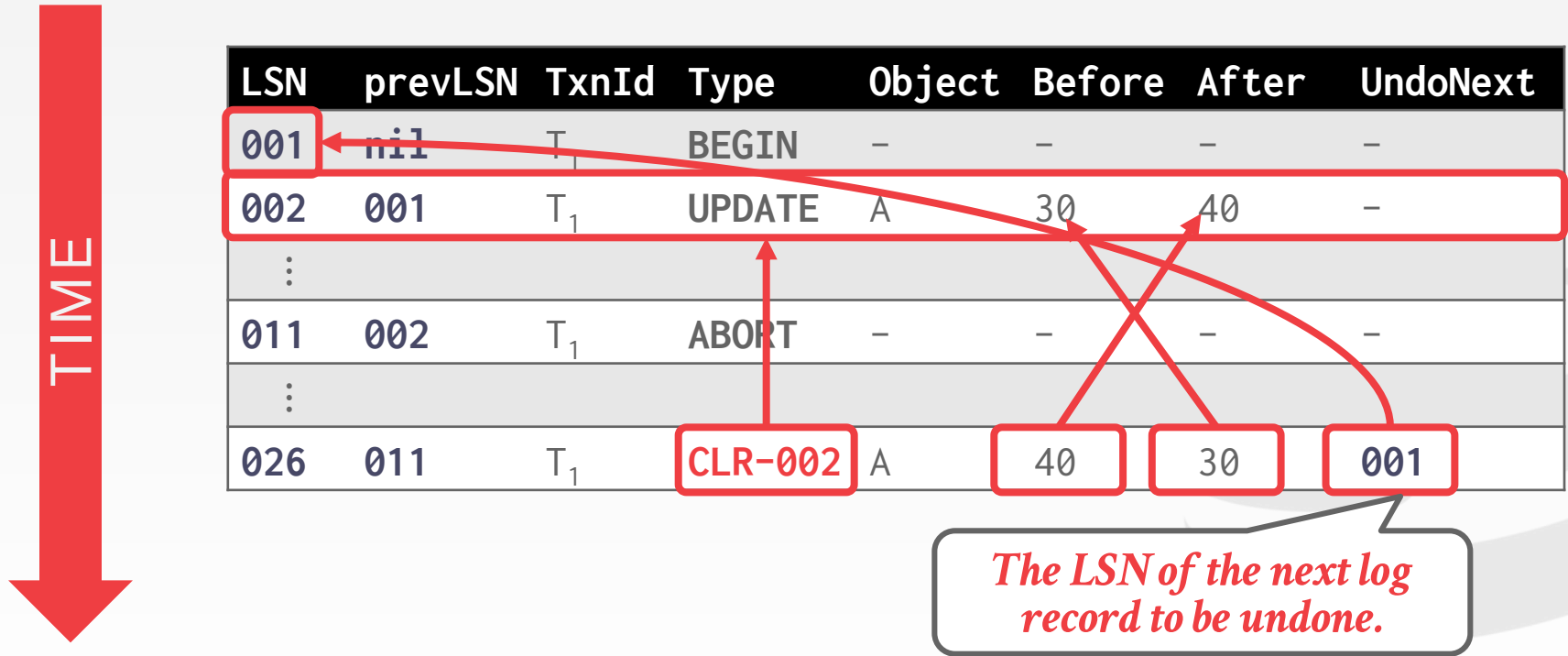
CLRs are added to log like any other record.

TRANSACTION ABORT – CLR EXAMPLE



LSN	prevLSN	TxnId	Type	Object	Before	After	UndoNext
001	nil	T_1	BEGIN	–	–	–	–
002	001	T_1	UPDATE	A	30	40	–
⋮							
011	002	T_1	ABORT	–	–	–	–

TRANSACTION ABORT – CLR EXAMPLE



TRANSACTION ABORT – CLR EXAMPLE



LSN	prevLSN	TxnId	Type	Object	Before	After	UndoNext
001	nil	T_1	BEGIN	–	–	–	–
002	001	T_1	UPDATE	A	30	40	–
⋮							
011	002	T_1	ABORT	–	–	–	–
⋮							
026	011	T_1	CLR-002	A	40	30	001
027	026	T_1	TXN-END	–	–	–	nil

ABORT ALGORITHM

First write an **ABORT** record to log for the txn.

Then play back the txn's updates in reverse order.

For each update record:

- Write a **CLR** entry to the log.
- Restore old value.

At end, write a **TXN-END** log record.

Notice: **CLRs** never need to be undone.



TODAY'S AGENDA

~~Log Sequence Numbers~~

~~Normal Commit & Abort Operations~~

Fuzzy Checkpointing

Recovery Algorithm



NON-FUZZY CHECKPOINTS

The DBMS halts everything when it takes a checkpoint to ensure a consistent snapshot:

- Halt the start of any new txns.
- Wait until all active txns finish executing.
- Flushes dirty pages on disk.

This is obviously bad...



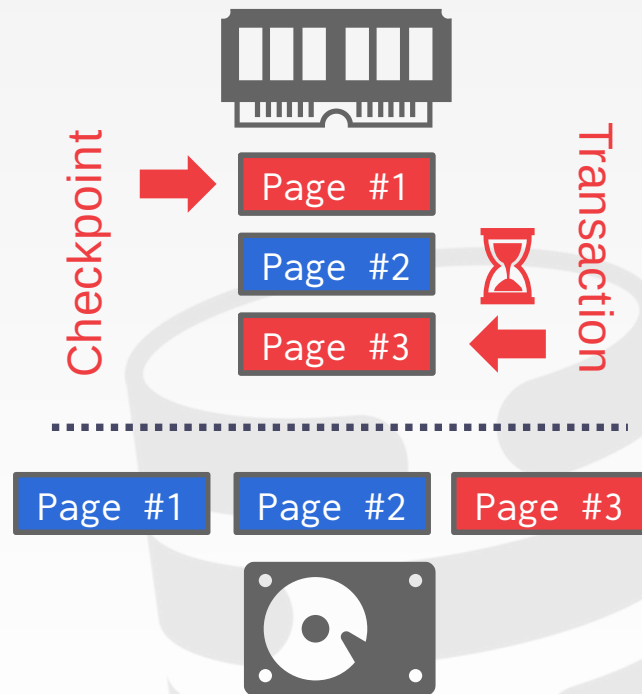
SLIGHTLY BETTER CHECKPOINTS

Pause modifying txns while the DBMS takes the checkpoint.

- Prevent queries from acquiring write latch on table/index pages.
- Don't have to wait until all txns finish before taking the checkpoint.

We must record internal state as of the beginning of the checkpoint.

- **Active Transaction Table (ATT)**
- **Dirty Page Table (DPT)**



ACTIVE TRANSACTION TABLE

One entry per currently active txn.

- **txnId**: Unique txn identifier.
- **status**: The current "mode" of the txn.
- **lastLSN**: Most recent *LSN* created by txn.

Entry removed when txn commits or aborts.

Txn Status Codes:

- **R** → Running
- **C** → Committing
- **U** → Candidate for Undo

DIRTY PAGE TABLE

Keep track of which pages in the buffer pool contain changes from uncommitted transactions.

One entry per dirty page in the buffer pool:

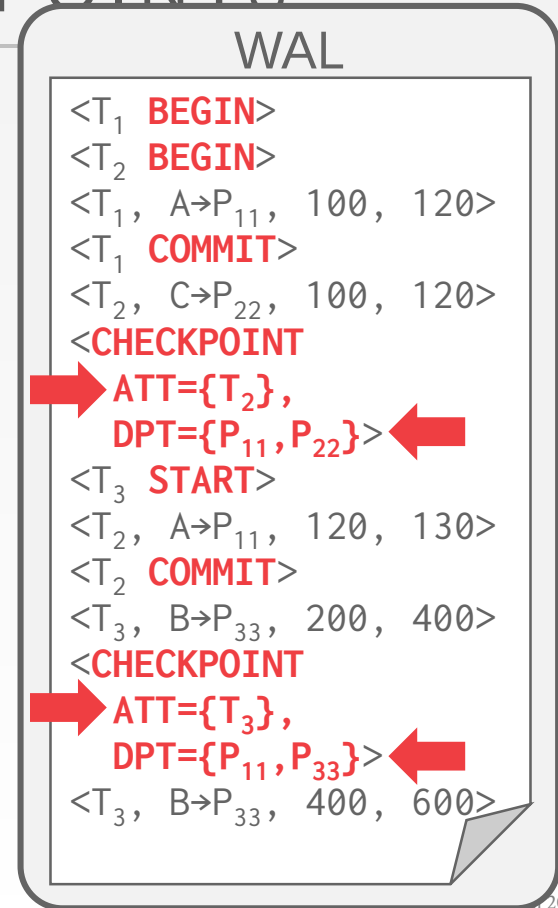
→ **recLSN**: The *LSN* of the log record that first caused the page to be dirty.

SLIGHTLY BETTER CHECKPOINTS

At the first checkpoint, T_2 is still running and there are two dirty pages (P_{11} , P_{22}).

At the second checkpoint, T_3 is active and there are two dirty pages (P_{11} , P_{33}).

This still is not ideal because the DBMS must stall txns during checkpoint...



FUZZY CHECKPOINTS

A *fuzzy checkpoint* is where the DBMS allows active txns to continue the run while the system flushes dirty pages to disk.

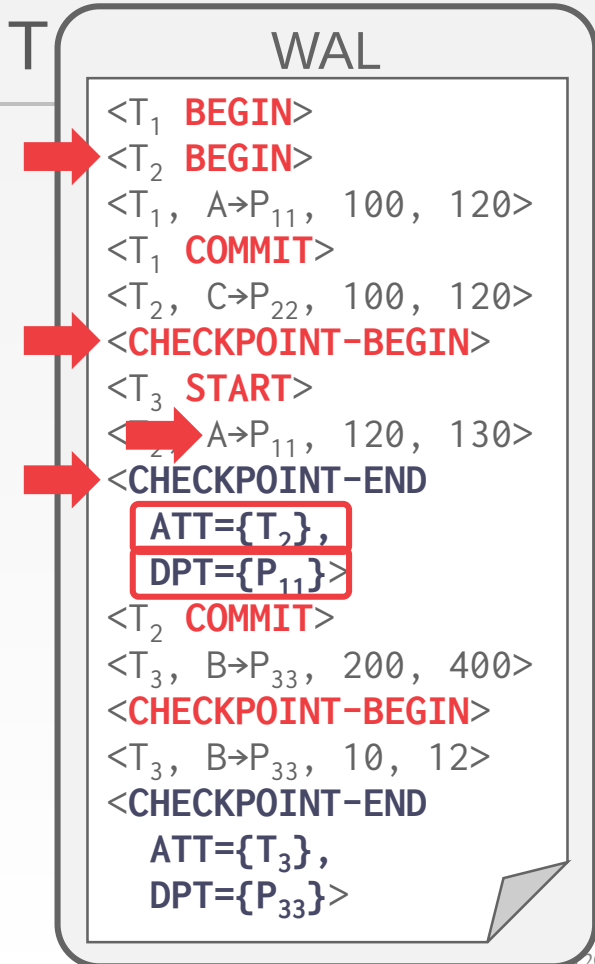
New log records to track checkpoint boundaries:

- **CHECKPOINT-BEGIN**: Indicates start of checkpoint
- **CHECKPOINT-END**: Contains **ATT** + **DPT**.

FUZZY CHECKPOINT

The *LSN* of the **CHECKPOINT-BEGIN** record is written to the database's **MasterRecord** entry on disk when the checkpoint successfully completes.

Any txn that starts after the checkpoint is excluded from the ATT in the **CHECKPOINT-END** record.



ARIES – RECOVERY PHASES

Phase #1 – Analysis

- Read WAL from last checkpoint to identify dirty pages in the buffer pool and active txns at the time of the crash.

Phase #2 – Redo

- Repeat all actions starting from an appropriate point in the log (even txns that will abort).

Phase #3 – Undo

- Reverse the actions of txns that did not commit before the crash.

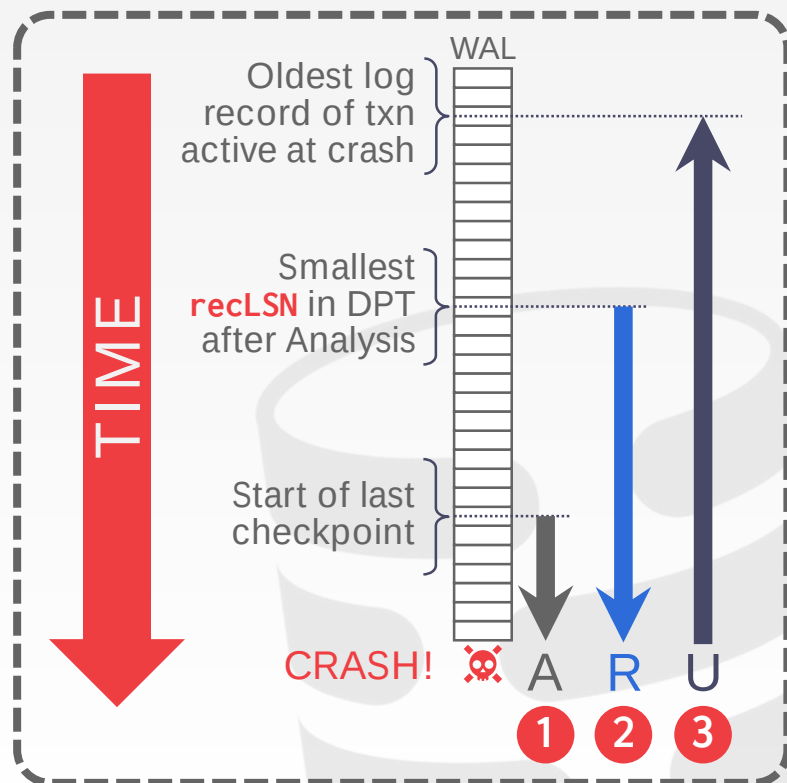
ARIES – OVERVIEW

Start from last **BEGIN-CHECKPOINT** found via **MasterRecord**.

Analysis: Figure out which txns committed or failed since checkpoint.

Redo: Repeat all actions.

Undo: Reverse effects of failed txns.



ANALYSIS PHASE

Scan log forward from last successful checkpoint.

If you find a **TXN-END** record, remove its corresponding txn from **ATT**.

All other records:

- Add txn to **ATT** with status **UNDO**.
- On commit, change txn status to **COMMIT**.

For **UPDATE** records:

- If page **P** not in **DPT**, add **P** to **DPT**, set its **recLSN=LSN**.

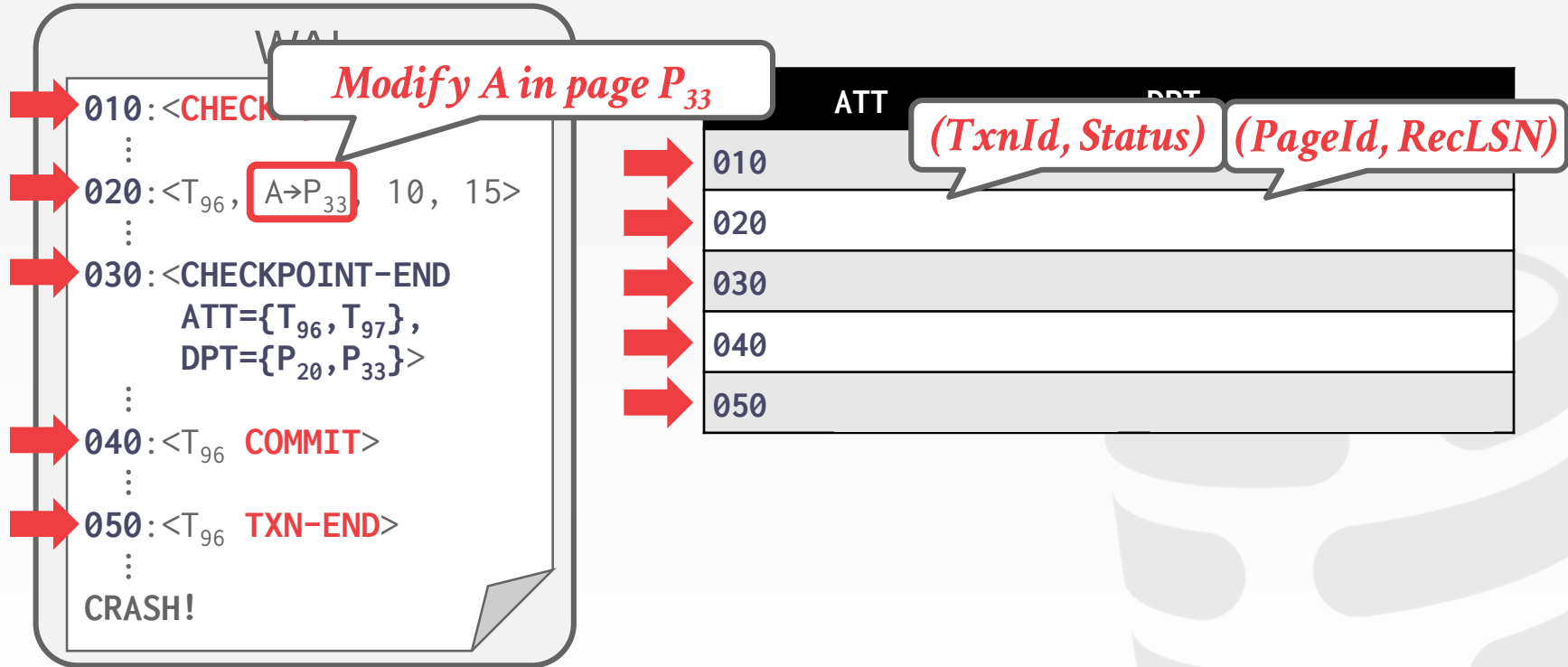
ANALYSIS PHASE

At end of the Analysis Phase:

- **ATT** tells the DBMS which txns were active at time of crash.
- **DPT** tells the DBMS which dirty pages might not have made it to disk.



ANALYSIS PHASE EXAMPLE



REDO PHASE

The goal is to repeat history to reconstruct state at the moment of the crash:

→ Reapply all updates (even aborted txns!) and redo **CLRs**.

There techniques that allow the DBMS to avoid unnecessary reads/writes, but we will ignore that in this lecture...

REDO PHASE

Scan forward from the log record containing smallest **recLSN** in **DPT**.

For each update log record or **CLR** with a given **LSN**, redo the action unless:

- Affected page is not in **DPT**, or
- Affected page is in **DPT** but that record's **LSN** is less than the page's **recLSN**.

REDO PHASE

To redo an action:

- Reapply logged action.
- Set **pageLSN** to log record's *LSN*.
- No additional logging, no forced flushes!

At the end of Redo Phase, write **TXN-END** log records for all txns with status **C** and remove them from the **ATT**.

UNDO PHASE

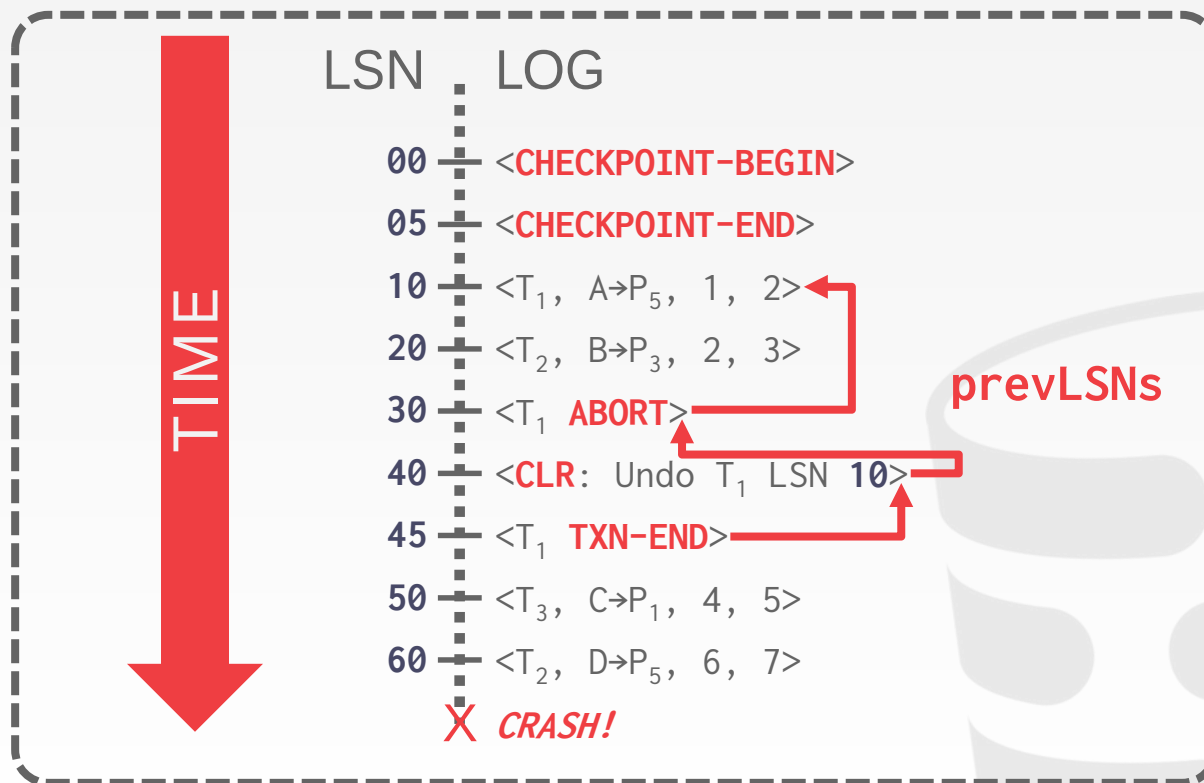
Undo all txns that were active at the time of crash and therefore will never commit.

→ These are all the txns with **U** status in the **ATT** after the Analysis Phase.

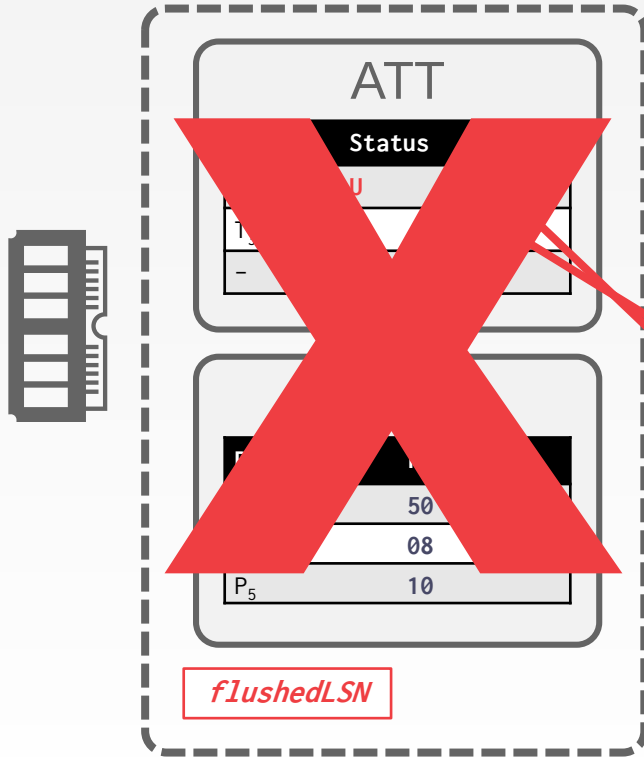
Process them in reverse **LSN** order using the **lastLSN** to speed up traversal.

Write a **CLR** for every modification.

FULL EXAMPLE



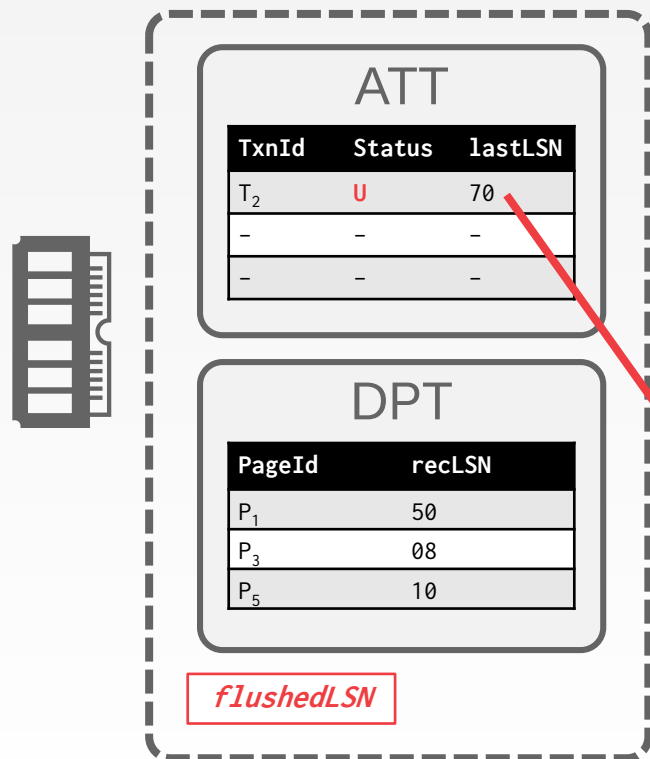
FULL EXAMPLE



LSN	LOG
00,05	<CHECKPOINT-BEGIN>, <CHECKPOINT-END>
10	<T ₁ , A→P ₅ , 1, 2>
20	<T ₂ , B→P ₃ , 2, 3>
30	<T ₁ ABORT>
40,45	<CLR: Undo T ₁ LSN 10>, <T ₁ TXN-END>
50	<T ₃ , C→P ₁ , 4, 5>
60	<T ₂ , D→P ₅ , 6, 7>
	X CRASH! RESTART!
70	<CLR: Undo T ₂ LSN 60, UndoNext>
80,85	<CLR: Undo T ₃ LSN 50>, <T ₃ TXN-END>
	X CRASH! RESTART!

*Flush dirty pages +
WAL to disk!*

FULL EXAMPLE



LSN	LOG
00,05	<CHECKPOINT-BEGIN>, <CHECKPOINT-END>
10	<T ₁ , A→P ₅ , 1, 2>
20	<T ₂ , B→P ₃ , 2, 3>
30	<T ₁ ABORT>
40,45	<CLR: Undo T ₁ LSN 10>, <T ₁ TXN-END>
50	<T ₃ , C→P ₁ , 4, 5>
60	<T ₂ , D→P ₅ , 6, 7>
	✗ CRASH! RESTART!
70	<CLR: Undo T ₂ LSN 60, UndoNext 20>
80,85	<CLR: Undo T ₃ LSN 50>, <T ₃ TXN-END>
	✗ CRASH! RESTART!
90,95	<CLR: Undo T ₂ LSN 20>, <T ₂ TXN-END>

ADDITIONAL CRASH ISSUES (1)

What does the DBMS do if it crashes during recovery in the Analysis Phase?

→ Nothing. Just run recovery again.

What does the DBMS do if it crashes during recovery in the Redo Phase?

→ Again nothing. Redo everything again.

ADDITIONAL CRASH ISSUES (2)

How can the DBMS improve performance during recovery in the Redo Phase?

- Assume that it is not going to crash again and flush all changes to disk asynchronously in the background.

How can the DBMS improve performance during recovery in the Undo Phase?

- Lazily rollback changes before new txns access pages.
- Rewrite the application to avoid long-running txns.

CONCLUSION

Mains ideas of ARIES:

- WAL with **STEAL/NO-FORCE**
- Fuzzy Checkpoints (snapshot of dirty page ids)
- Redo everything since the earliest dirty page
- Undo txns that never commit
- Write **CLRs** when undoing, to survive failures during restarts

Log Sequence Numbers:

- **LSNs** identify log records; linked into backwards chains per transaction via **prevLSN**.
- **pageLSN** allows comparison of data page and log records.

NEXT CLASS

You now know how to build a single-node DBMS.

So now we can talk about distributed databases!



CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL
数据库

聚合
内存管理
树索引

排序
查询规划
索引构建

并发控制
查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击**底部菜单栏**
称为 **AI 内容创作者**？回复 [添砖加瓦]